will be handy in case if you want to check that the data doesn't exist in the database. The advantage of these methods is that you are free to work with the integrated assertion methods included with PHPUnit.

To create a factory inside the `database/factories` directory, you should use the make:factory command as shown below:

```
php artisan make:factory PostFactory
```

You can also use `--model` option to denote the name of the model created by the factory such as `--model=Post` embedded to the end of the above command. It is possible to reset the database after each test with the help of the `RefreshDatabase` trait irrespective of whether you are using a conventional database or an in-memory database.

The beauty of Laravel is that the framework enables you to define a default set of attributes for your Eloquent models via model factories. The `database/factories/UserFactory.php` file mainly contains one factory related definition in the form of a Faker PHP library. You can easily generate several random data with various possibilities for testing purposes. The UserFactory.php and CommentFactory.php files can also be created inside your database/factories directory for improved organisation of test data. There is no need to worry because these files are loaded by default by the framework.

If you work with model factories, you should be aware about factory states. You can use the `state()` method by passing an array to create and define your state transformations. A closure attribute can also be used if your state requires some sort of calculations or a $faker instance.

The `factory()` function available globally should be handed over the required task as soon as you defined your factories. It is possible to create models using `make()` method or generate a collection of several models by passing the required attribute inside factory() method. You can override attributes by passing an array of values to the make() method. The model instances can be generated using the create() method. It also saves itself to the database with the help of the save() method.

If you are indulged in the PHPUnit tests via Laravel, the framework itself provides methods such as `assertDatabaseHas(), assertDatabaseMissing()` and `assertSoftDeleted()` to simplify your tests.

Mocking

During the testing process of your Laravel applications, you will have to silence certain modules in such a way that they are not executed during